

ComicCraft

Complete AI-Powered Comic Generation Project – Technical Implementation Guide

This document elaborates the ComicCraft project structure, backend, Gemini story/JSON generation, Hugging Face image generation, responsive frontend, panel preview, PDF export, environment configuration, testing, and VS Code execution workflow.

1. Project Overview

ComicCraft is an AI-powered web application that converts a user's story idea into a multi-panel comic. The application combines a responsive HTML/CSS/JavaScript frontend with a FastAPI backend. Gemini is used to generate a structured story and panel-specific prompts. Hugging Face's InferenceClient text_to_image interface is used to generate illustrations. The generated panels are displayed in the browser and can be exported into a PDF using FPDF2.

2. System Architecture

The application follows a layered architecture. The frontend collects the user's topic and number of panels. FastAPI receives the request and coordinates the AI services. Gemini creates structured JSON containing the title, scene, dialogue, and image prompt for each panel. Each image prompt is passed to Hugging Face for image generation. The resulting images and story information are returned to the frontend. Finally, FPDF2 creates a printable PDF containing the comic panels and text.

3. Recommended Folder Structure

```
ComicCraft/  
app/  
__init__.py  
main.py  
config.py  
models.py  
services/  
gemini_service.py  
image_service.py  
pdf_service.py  
templates/  
index.html  
static/  
css/style.css  
js/app.js  
generated/  
tests/  
test_main.py  
test_gemini.py  
test_pdf.py  
.env.example  
.gitignore  
requirements.txt  
README.md
```

4. FastAPI Backend – app/main.py

FastAPI acts as the central application layer. It serves the frontend, exposes the comic-generation endpoint, calls the Gemini and image-generation services, and provides a PDF-export endpoint.

Code – app/main.py

```
from fastapi import FastAPI, HTTPException
from fastapi.staticfiles import StaticFiles
from fastapi.responses import FileResponse

from app.models import StoryRequest
from app.services.gemini_service import generate_story
from app.services.image_service import generate_image
from app.services.pdf_service import create_pdf

app = FastAPI(
    title="ComicCraft API",
    description="AI-powered comic story generator",
    version="1.0.0"
)

app.mount("/static", StaticFiles(directory="app/static"), name="static")

@app.get("/")
async def home():
    return FileResponse("app/templates/index.html")

@app.post("/api/generate")
async def generate_comic(request: StoryRequest):
    try:
        story = await generate_story(request.topic, request.panels)

        for panel in story["panels"]:
            image_path = await generate_image(
                panel["image_prompt"],
                panel["panel_number"]
            )
            panel["image"] = image_path

        return story

    except Exception as error:
        raise HTTPException(status_code=500, detail=str(error))

@app.post("/api/export-pdf")
async def export_pdf(story: dict):
    pdf_path = create_pdf(story)
    return {"message": "PDF created successfully", "file": pdf_path}
```

5. Request Model – app/models.py

Pydantic is used to validate incoming API requests. The topic must contain at least three characters and the number of panels is restricted to a practical range.

Code – app/models.py

```
from pydantic import BaseModel, Field

class StoryRequest(BaseModel):
    topic: str = Field(..., min_length=3, max_length=500)
    panels: int = Field(default=6, ge=1, le=12)
```

6. Gemini Story and JSON Generation

Gemini converts the user's natural-language idea into predictable structured data. Each panel contains a panel number, scene description, dialogue, and image-generation prompt. Returning JSON makes it possible for the backend to process every panel independently.

Code – app/services/gemini_service.py

```
import os
```

```

import json
from google import genai

client = genai.Client(api_key=os.getenv("GEMINI_API_KEY"))

async def generate_story(topic: str, panels: int):
    prompt = f'''
Create a short comic story based on:
Topic: {topic}
Number of panels: {panels}

Return ONLY valid JSON using this structure:
{{
  "title": "Comic title",
  "description": "Short description",
  "panels": [
    {{
      "panel_number": 1,
      "scene": "Description of the scene",
      "dialogue": "Character dialogue",
      "image_prompt": "Detailed image generation prompt"
    }}
  ]
}}

Make sure there are exactly {panels} panels.
'''

    response = client.models.generate_content(
        model="gemini-2.5-flash",
        contents=prompt
    )

    text = response.text.strip()

    if text.startswith("```"):
        text = text.replace("```json", "")
        text = text.replace("```", "").strip()

    return json.loads(text)

```

7. Hugging Face Image Generation

The image-generation service receives the `image_prompt` produced by Gemini. Hugging Face's `InferenceClient.text_to_image()` generates the illustration and the application saves it as a PNG file under the static generated directory.

Code – `app/services/image_service.py`

```

import os
from pathlib import Path
from huggingface_hub import InferenceClient

client = InferenceClient(
    provider="hf-inference",
    api_key=os.getenv("HF_TOKEN")
)

OUTPUT_DIR = Path("app/static/generated")
OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

async def generate_image(prompt: str, panel_number: int):
    image = client.text_to_image(
        prompt=prompt,
        model="stabilityai/stable-diffusion-xl-base-1.0"
    )

    file_path = OUTPUT_DIR / f"panel_{panel_number}.png"
    image.save(file_path)

    return f"/static/generated/panel_{panel_number}.png"

```

8. Frontend – `index.html`

The frontend provides a simple interface where the user enters a story idea and selects the number of panels. JavaScript sends the request to FastAPI and renders the returned comic dynamically.

Code – app/templates/index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>ComicCraft</title>
  <link rel="stylesheet" href="/static/css/style.css">
</head>
<body>
  <header>
    <h1>ComicCraft</h1>
    <p>Create AI-powered comics from your ideas</p>
  </header>

  <main>
    <section class="generator">
      <label>Enter your story idea</label>
      <textarea id="topic"
        placeholder="Example: A girl discovers a magical forest..."></textarea>

      <label>Number of panels</label>
      <input id="panels" type="number" min="1" max="12" value="6">

      <button onclick="generateComic()">Generate Comic</button>
    </section>

    <section id="loading" class="hidden">
      Generating your comic...
    </section>

    <section id="comic-container" class="comic-grid"></section>

    <button id="pdfButton" class="hidden" onclick="exportPDF()">
      Download PDF
    </button>
  </main>

  <script src="/static/js/app.js"></script>
</body>
</html>
```

9. Responsive CSS

The CSS uses a responsive grid so panels automatically adapt to desktop, tablet, and mobile screens.

Code – app/static/css/style.css

```
* {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}

body {
  font-family: Arial, sans-serif;
  background: #f4f4f4;
  padding: 20px;
}

header {
  text-align: center;
  margin-bottom: 30px;
}

.generator {
  max-width: 700px;
  margin: auto;
  background: white;
  padding: 25px;
  border-radius: 12px;
}

textarea, input {
  width: 100%;
  padding: 12px;
  margin: 10px 0 20px;
}
```

```

button {
  padding: 12px 20px;
  border: none;
  border-radius: 8px;
  cursor: pointer;
}

.comic-grid {
  max-width: 1100px;
  margin: 30px auto;
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(280px, 1fr));
  gap: 20px;
}

.panel {
  background: white;
  border-radius: 12px;
  overflow: hidden;
  box-shadow: 0 4px 12px rgba(0,0,0,0.1);
}

.panel img {
  width: 100%;
  display: block;
}

.panel-content {
  padding: 15px;
}

.hidden {
  display: none;
}

@media (max-width: 600px) {
  body { padding: 10px; }
  .generator { padding: 15px; }
  .comic-grid { grid-template-columns: 1fr; }
}

```

10. JavaScript – Panel-by-Panel Preview

JavaScript communicates with the FastAPI API using `fetch()`. Once the JSON response is received, each panel is converted into an HTML card containing the generated image, scene, and dialogue.

Code – `app/static/js/app.js`

```

let currentStory = null;

async function generateComic() {
  const topic = document.getElementById("topic").value;
  const panels = Number(document.getElementById("panels").value);

  if (!topic.trim()) {
    alert("Please enter a story idea.");
    return;
  }

  document.getElementById("loading").classList.remove("hidden");

  try {
    const response = await fetch("/api/generate", {
      method: "POST",
      headers: {"Content-Type": "application/json"},
      body: JSON.stringify({topic, panels})
    });

    if (!response.ok) {
      throw new Error("Comic generation failed");
    }

    currentStory = await response.json();
    displayComic(currentStory);

  } catch (error) {
    console.error(error);
    alert("Unable to generate comic.");
  } finally {

```

```

        document.getElementById("loading").classList.add("hidden");
    }
}

function displayComic(story) {
    const container = document.getElementById("comic-container");
    container.innerHTML = "";

    story.panels.forEach(panel => {
        const panelElement = document.createElement("article");
        panelElement.className = "panel";

        panelElement.innerHTML = `
            
            <div class="panel-content">
                <h3>Panel ${panel.panel_number}</h3>
                <p>${panel.scene}</p>
                <strong>${panel.dialogue}</strong>
            </div>
        `;

        container.appendChild(panelElement);
    });

    document.getElementById("pdfButton")
        .classList.remove("hidden");
}

```

11. PDF Export using FPDF2

FPDF2 creates a PDF document containing a title page followed by individual comic-panel pages. The generated PNG image is inserted into the PDF and the panel scene and dialogue are written underneath.

Code – app/services/pdf_service.py

```

from pathlib import Path
from fpdf import FPDF

def create_pdf(story: dict):
    output_dir = Path("app/static/generated")
    output_dir.mkdir(parents=True, exist_ok=True)

    pdf_path = output_dir / "comiccraft_comic.pdf"

    pdf = FPDF()
    pdf.set_auto_page_break(auto=True, margin=15)

    pdf.add_page()
    pdf.set_font("Arial", "B", 24)
    pdf.cell(0, 20, story["title"], align="C",
            new_x="LMARGIN", new_y="NEXT")

    for panel in story["panels"]:
        pdf.add_page()

        pdf.set_font("Arial", "B", 16)
        pdf.cell(0, 10, f"Panel {panel['panel_number']}",
            new_x="LMARGIN", new_y="NEXT")

        image_path = panel.get("image")

        if image_path:
            local_path = image_path.replace(
                "/static/", "app/static/"
            )
            pdf.image(local_path, x=15, y=35, w=180)

        pdf.set_y(145)
        pdf.set_font("Arial", "", 12)
        pdf.multi_cell(0, 8, panel["scene"])
        pdf.ln(5)
        pdf.multi_cell(0, 8, panel["dialogue"])

    pdf.output(str(pdf_path))
    return str(pdf_path)

```

12. Environment Configuration

API keys should never be hard-coded in source files. The project uses a .env file and python-dotenv to load secrets from environment variables.

Code – .env.example

```
GEMINI_API_KEY=your_gemini_api_key_here
HF_TOKEN=your_huggingface_token_here
```

Code – .gitignore

```
.env
.venv/
__pycache__/
*.pyc
app/static/generated/
```

Load Environment Variables

```
from dotenv import load_dotenv
load_dotenv()
```

13. Requirements

The requirements file contains the core backend framework, AI SDKs, PDF library, environment-variable loader, validation library, and testing tools.

Code – requirements.txt

```
fastapi
uvicorn[standard]
google-gemai
huggingface_hub
fpdf2
python-dotenv
pydantic
pytest
httpx
```

14. Testing

Automated tests help confirm that the API starts and that key components behave as expected. A basic FastAPI test can verify the home page response.

Code – tests/test_main.py

```
from fastapi.testclient import TestClient
from app.main import app

client = TestClient(app)

def test_home_page():
    response = client.get("/")
    assert response.status_code == 200
```

Compile Check

```
python -m compileall app
```

Run Tests

```
pytest
```

15. VS Code Setup and Execution

1. Extract the ComicCraft project.
2. Open the ComicCraft folder in VS Code.

3. Open the integrated terminal.
4. Create the virtual environment.
5. Activate the environment.
6. Install dependencies.
7. Copy .env.example to .env.
8. Add the Gemini API key and Hugging Face token.
9. Start the FastAPI server.
10. Open the local URL in a browser.

Commands – Windows

```
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
uvicorn app.main:app --reload
```

Browser URL

<http://127.0.0.1:8000>

16. Complete Data Flow

User enters a story idea

|

v

Frontend sends POST /api/generate

|

v

FastAPI validates the request

|

v

Gemini generates structured JSON

|

v

Each image_prompt is extracted

|

v

Hugging Face generates an illustration

|

v

PNG images are stored

|

v

FastAPI returns story + image paths

|

v

JavaScript renders panel-by-panel preview

|

v

FPDF2 creates final PDF

17. Technical Explanation for Project Report

ComicCraft uses a modular AI-based architecture for automated comic creation. The frontend is implemented using HTML, CSS, and JavaScript to provide a responsive user interface and panel-by-panel preview. FastAPI

provides REST-based backend endpoints and coordinates communication between the frontend and external AI services. Google's modern google-genai SDK is used to communicate with Gemini for structured story generation. The generated JSON contains panel-level scene descriptions, dialogue, and image prompts. These prompts are passed to Hugging Face's InferenceClient.text_to_image() interface to produce illustrations using a Stable Diffusion-compatible image-generation model. Generated images are stored in the application's static directory and displayed dynamically in the browser. FPDF2 is used to combine the generated artwork and textual content into a downloadable PDF. Environment variables are used to protect API credentials, while pytest and Python compilation checks support basic software validation.

18. Advantages of the Implementation

- Modular architecture makes individual services easier to maintain.
- Structured JSON allows reliable panel-by-panel processing.
- AI-generated image prompts can be customized for different comic themes.
- Responsive frontend supports different screen sizes.
- FastAPI provides lightweight and fast API handling.
- PDF export makes the generated comic easy to save and share.
- Environment variables keep API credentials outside the source code.
- Automated tests and compilation checks help identify implementation errors early.

19. Conclusion

ComicCraft demonstrates how generative AI can be integrated into a complete web application. The project combines natural-language story generation, AI image generation, web development, API integration, and document generation into one workflow. Its modular structure allows the story engine, image-generation engine, frontend, and PDF module to be developed and tested independently while working together as a single comic-generation platform.